

Artificial Intelligence — Lecture 3

Local Search, Optimization, and Search Performance

Dr. Fayçal Hamdi

Faculty of Computer and Information Systems

Lecture 3 at a Glance

Dr. Fayçal Hamdi - For Course Use Only - 2026

Local Search, Optimization, and Search Performance

Course Learning Outcome

CLO3: Illustrate the performance of local, global, and optimal search strategies.

Lecture purpose

- Explain why some AI problems are better viewed as **optimization problems** rather than path-finding problems.
- Compare **local search**, **systematic/global search**, and **optimal search** in terms of guarantees and practical performance.
- Study hill climbing, simulated annealing, local beam search, and genetic algorithms.

Conceptual structure

Stage	Core question
Optimization view	What makes a state better than another state?
Local improvement	How can we improve a candidate without storing full paths?
Escaping traps	How can search avoid local optima, plateaus, and ridges?
Performance comparison	Which method is suitable under which assumptions?

💡 Main idea: local search usually cares about the **quality of the current state**, not the path used to reach it.

Learning Objectives

Dr. Fayçal Hamdi - For Course Use Only - 2026

By the end of Lecture 3, you will be able to:

1. Distinguish **path-finding search** from **optimization-oriented local search**.
2. Define objective functions, neighborhood functions, local optima, global optima, plateaus, and ridges.
3. Apply hill climbing and explain why it can fail.
4. Explain how random restarts and simulated annealing help escape poor local optima.
5. Describe local beam search and genetic algorithms as population-based search strategies.
6. Compare local, systematic/global, and optimal search strategies using performance criteria.
7. Select an appropriate search method for representative AI problem types.

Lecture Roadmap

Dr. Fayçal Hamdi - For Course Use Only - 2026

From paths to optimization landscapes

Stage	Main idea	Student output
1. Optimization formulation	A state has a measurable quality or cost.	Formalize a problem using states, neighbors, and an objective function.
2. Hill climbing	Repeatedly move to a better neighbor.	Trace a local improvement process and identify failure modes.
3. Escaping local traps	Randomness and temperature can avoid premature convergence.	Explain random-restart hill climbing and simulated annealing.
4. Population-based search	Multiple candidates are improved together.	Compare local beam search and genetic algorithms.
5. Performance comparison	Search choice depends on guarantees, memory, and solution quality.	Justify the algorithm choice for a problem scenario.

Part I

Dr. Fayçal Hamdi — Course Use Only - 2026

Optimization View and Local Improvement

Classical search asks: **Which path reaches a goal?** Local search often asks: **Which state has the best value?**

Dr. Fayçal Hamdi — For Course Use Only - 2026

From Path Search to Optimization

Some problems do not require the path itself

Path-finding search

A solution is an action sequence:

$$[a_1, a_2, \dots, a_k]$$

The path matters because we want to know **how to reach** the goal.

Route finding, maze navigation, puzzle solving, and planning usually require a path.

Optimization search

A solution is a good or best state:

$$s^* = \arg \max_{s \in S} V(s)$$

or equivalently a minimum-cost state.

Timetabling, N-Queens, feature selection, layout design, and parameter tuning often care mainly about solution quality.

Optimization Problem Formalization

States, neighbors, and objective values

A local search problem can be described using:

$$P_{local} = (S, N, V)$$

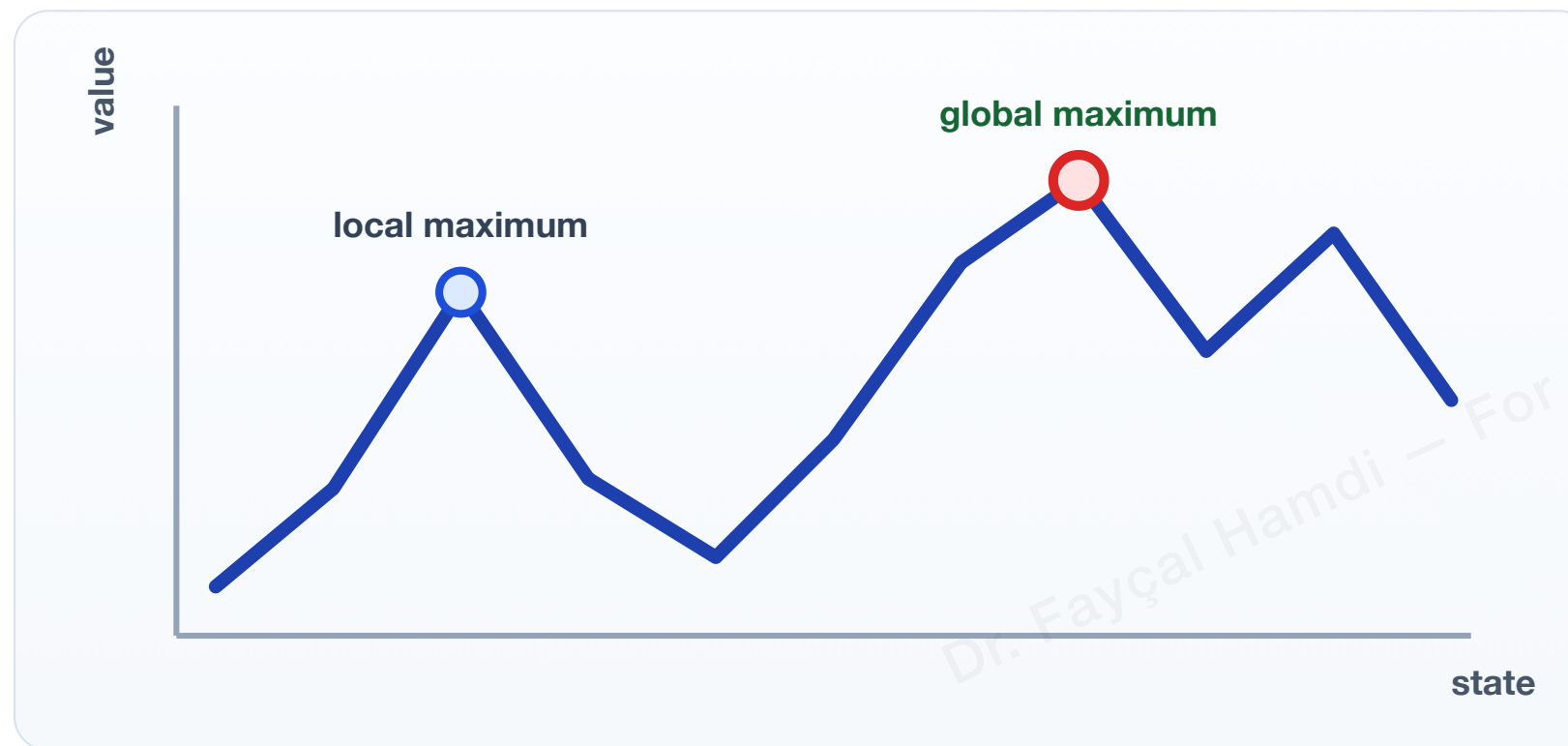
where:

Component	Meaning	Example
S	Set of complete candidate states	A complete N-Queens board configuration
$N(s)$	Neighborhood of state s	Boards obtained by moving one queen
$V(s)$	Objective or fitness value	Negative number of attacking queen pairs

The algorithm usually keeps one or more current states and repeatedly replaces them with better or more promising states.

Search Landscape

Objective values create peaks, valleys, plateaus, and ridges



Key concepts

- A **local optimum** is better than neighboring states.
- A **global optimum** is best among all states.
- A **plateau** contains many states with similar values.
- A **ridge** may require sideways or non-greedy movement.

Local search can perform very well, but it may stop at a local optimum that is not globally best.

Why Local Search Is Useful

Large state spaces can make systematic search impractical

Systematic search methods from Lecture 2 maintain a frontier of paths. This is powerful when path guarantees are needed, but it can become memory-intensive.

Local search is useful when:

- the state space is too large for exhaustive or frontier-based exploration,
- the path to the solution is irrelevant,
- a good solution is acceptable even if optimality is not guaranteed,
- the problem is naturally expressed as maximizing or minimizing an objective.

💡 Local search trades strong guarantees for practical scalability and low memory use.

Performance Criteria for Local Search

We evaluate more than only completeness and optimality

Criterion	Question	Why it matters
Solution quality	How good is the final state?	Many optimization tasks accept high-quality approximate solutions.
Convergence speed	How quickly does the search improve?	Fast improvement may be more useful than guaranteed optimality.
Memory use	How many states are stored?	Local search often stores only one or a few candidates.
Robustness	Does it avoid poor local optima?	Randomization and restarts can improve reliability.
Guarantee	Does it provably find a global optimum?	Some applications require optimality; many do not.

Hill-Climbing Search

Dr. Fayçal Hamdi - Course Use Only - 2026

Always move to a better neighbor

Main idea

Hill climbing starts from a current state and repeatedly moves to a neighboring state with a better objective value.

For maximization, the rule is:

$$s \leftarrow \arg \max_{s' \in N(s)} V(s')$$

when that neighbor improves the current value.

Important property

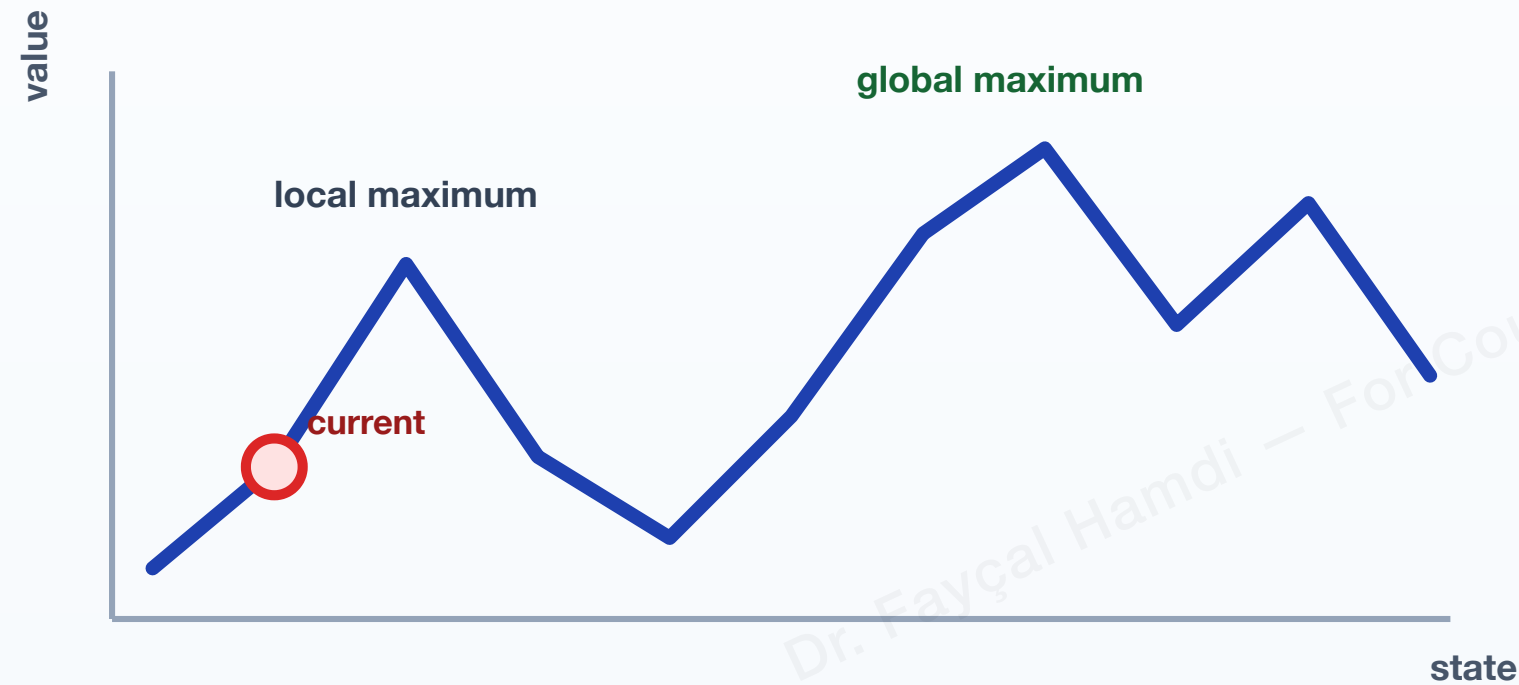
Hill climbing is **local**: it only compares the current state with nearby alternatives.

In N-Queens, a neighbor can be obtained by moving one queen to another row in the same column.

If no neighbor improves the value, the algorithm stops even if a better state exists far away.

Hill Climbing Walkthrough

A local method can stop before reaching the global optimum



Step 1 — initialize

Main idea

Start from an arbitrary state and evaluate its neighboring states.

Current state

state x1

Objective value

moderate value

Decision

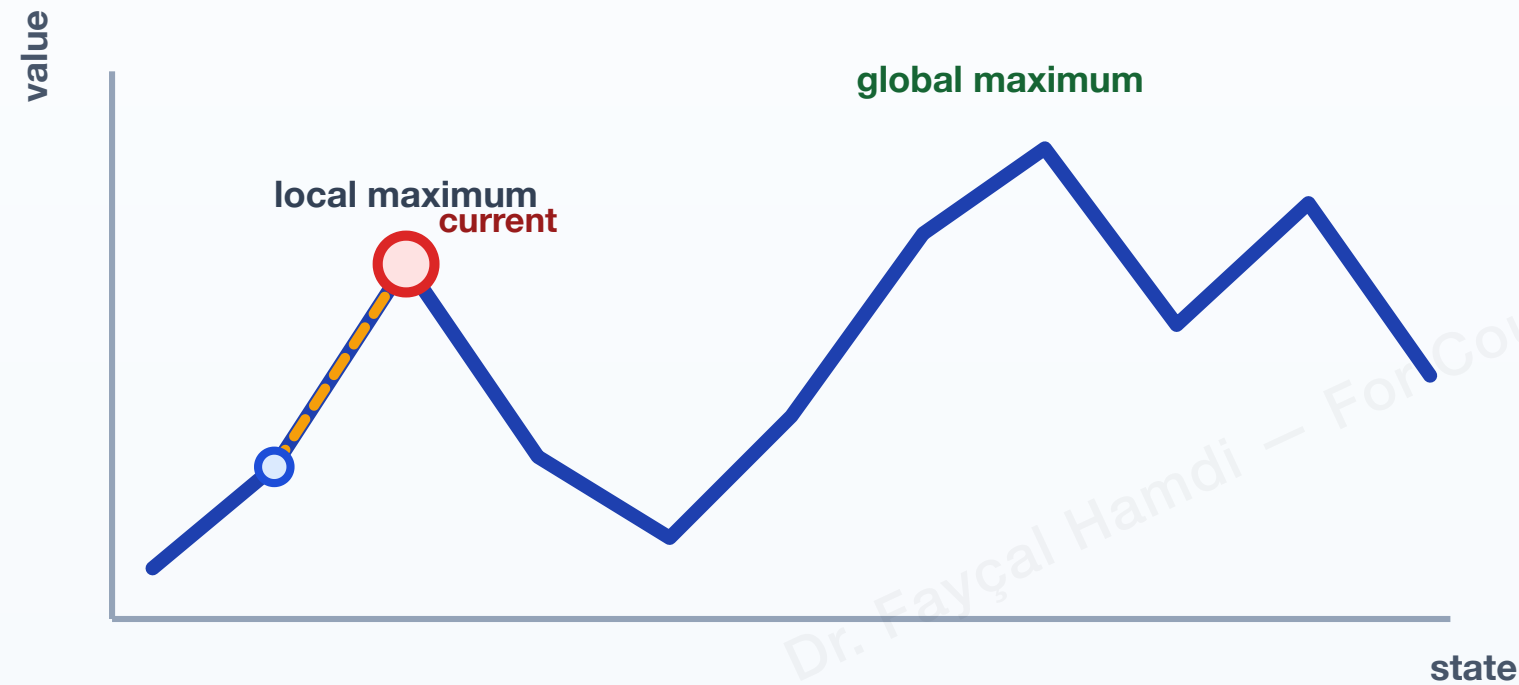
Move to the neighbor with the highest value.

Legend

current state

visited state

A local method can stop before reaching the global optimum



Step 2 — climb to a local maximum

Main idea

Move to the neighboring state with the highest value. At x_2 , neither neighbor provides an improvement.

Current state

state x_2

Objective value

higher than both immediate neighbors

Decision

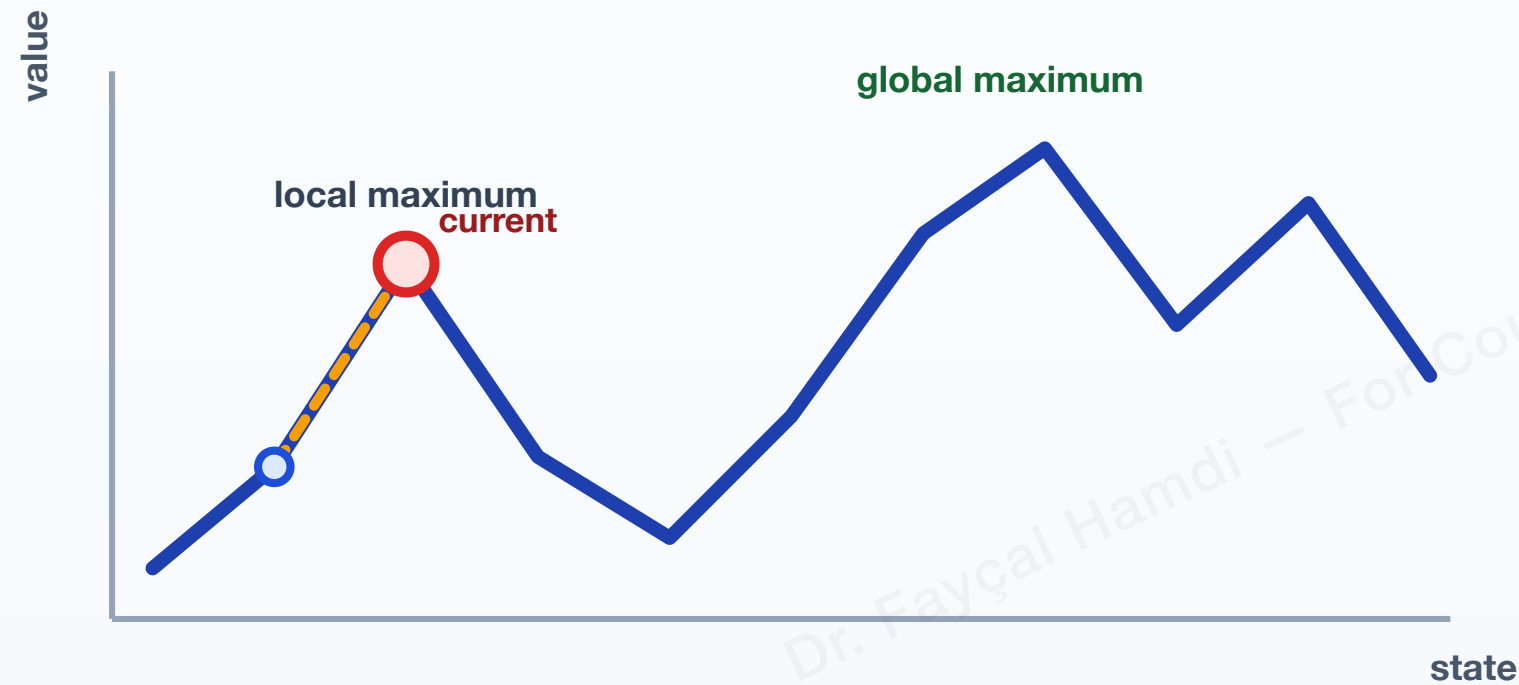
No neighboring state has a higher value.

Legend

current state

visited state

A local method can stop before reaching the global optimum



Step 3 — stop at local maximum

Main idea

Hill climbing cannot see beyond the valley. It stops even though a better region exists elsewhere.

Current state

state x2

Objective value

local maximum

Decision

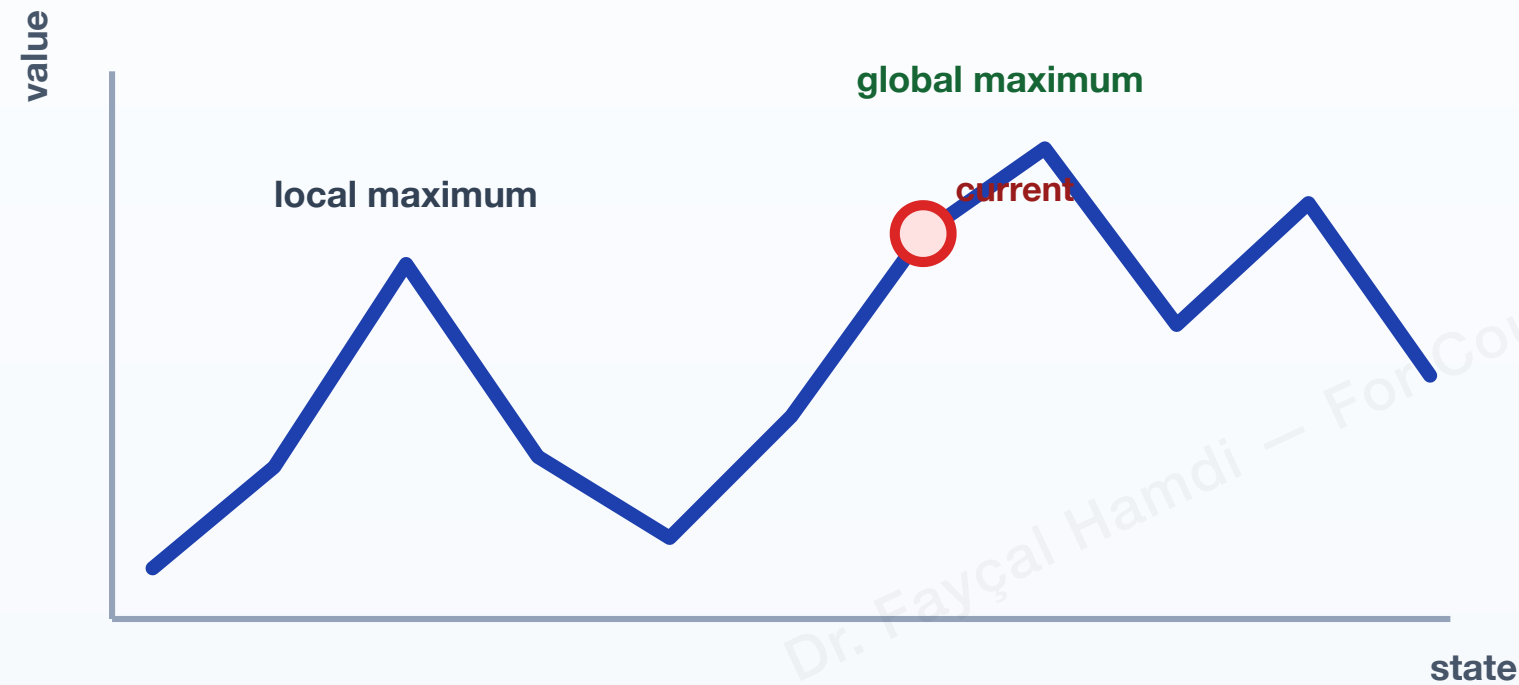
Return the current state; global optimality is not guaranteed.

Legend

current state

visited state

A local method can stop before reaching the global optimum



Step 4 — restart

Main idea

Random-restart hill climbing begins another run from a different initial state.

Current state

state x6

Objective value

promising region

Decision

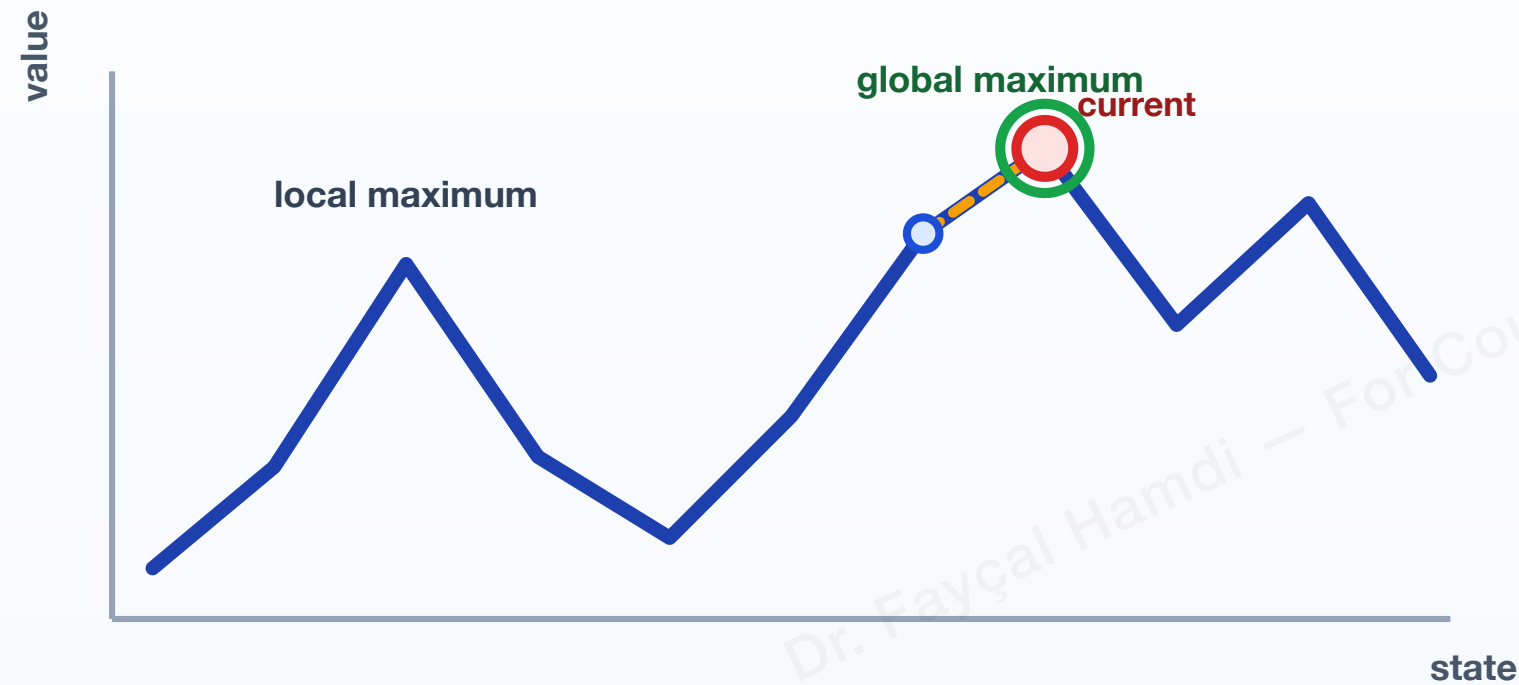
Restarting increases the probability of finding a better peak.

Legend

current state

visited state

A local method can stop before reaching the global optimum



Step 5 — better peak found

Main idea

The restarted run reaches the highest peak in the represented landscape.

Current state

state x7

Objective value

highest value in the illustrated landscape

Decision

Retain the best solution found across the restarts.

Legend

current state

visited state

best found

Hill-Climbing Search — Pseudocode

A simple local improvement loop

```
def hill_climbing(problem):  
    current = random_initial_state(problem)  
  
    while True:  
        neighbor = best_successor(current, problem)  
  
        if value(neighbor) <= value(current):  
            return current  
  
        current = neighbor
```

💡 The algorithm stores only the current state and its neighborhood, so memory use is usually very small.

Hill Climbing: Main Variants

Different ways to choose the next move

Variant	Selection rule	Main advantage	Main limitation
Steepest-ascent	Choose the best improving neighbor.	Strong local improvement.	Requires evaluating all neighbors.
First-choice	Choose the first improving neighbor found.	Useful for large neighborhoods.	May miss a better nearby move.
Stochastic	Randomly choose among improving moves.	Adds diversity.	Less predictable behavior.
Random-restart	Run hill climbing from multiple initial states.	Reduces local-optimum risk.	More runs require more time.

Failure Modes of Hill Climbing

Dr. Fayçal Hamdi - For Course Use Only - 2026

Why purely greedy local improvement can fail

Failure mode	Description	Typical consequence
Local maximum	State is better than all neighbors but not globally best.	Search stops too early.
Plateau	Many neighboring states have the same value.	Search may wander or stop.
Ridge	Progress requires a sequence of sideways or temporarily worse moves.	Greedy movement becomes inefficient.
Poor initial state	Starting point is in a bad region.	Final solution depends heavily on initialization.

Random restarts are often simple and effective: run local search multiple times and keep the best result.

Random-Restart Hill Climbing

Dr. Fayçal Hamdi - Course Use Only - 2026

Improve reliability by trying multiple starting points

Random-restart hill climbing repeats hill climbing from different initial states:

$$\text{best} = \max_{i=1}^r V(s_i^{\text{final}})$$

where r is the number of restarts.

```
def random_restart_hill_climbing(problem, restarts):  
    best = None  
    for i in range(restarts):  
        candidate = hill_climbing(problem)  
        if best is None or value(candidate) > value(best):  
            best = candidate  
    return best
```

💡 If local optima are numerous but good solutions are common, random restarts can be surprisingly effective.

Part II

Dr. Fayçal Hamdi - Course Use Only - 2026

Escaping Local Traps and Comparing Performance

To improve local search, we add mechanisms that maintain diversity, accept controlled risk, or search with multiple candidates.

Dr. Fayçal Hamdi — For Course Use Only - 2026

Simulated Annealing

Sometimes accept a worse move to escape a trap

Main idea

Simulated annealing is inspired by physical annealing. The algorithm starts with a high temperature and gradually cools.

For maximization, let:

$$\Delta = V(s') - V(s)$$

If $\Delta > 0$, accept the move. If $\Delta \leq 0$, accept it with probability:

$$P = e^{\Delta/T}$$

Interpretation

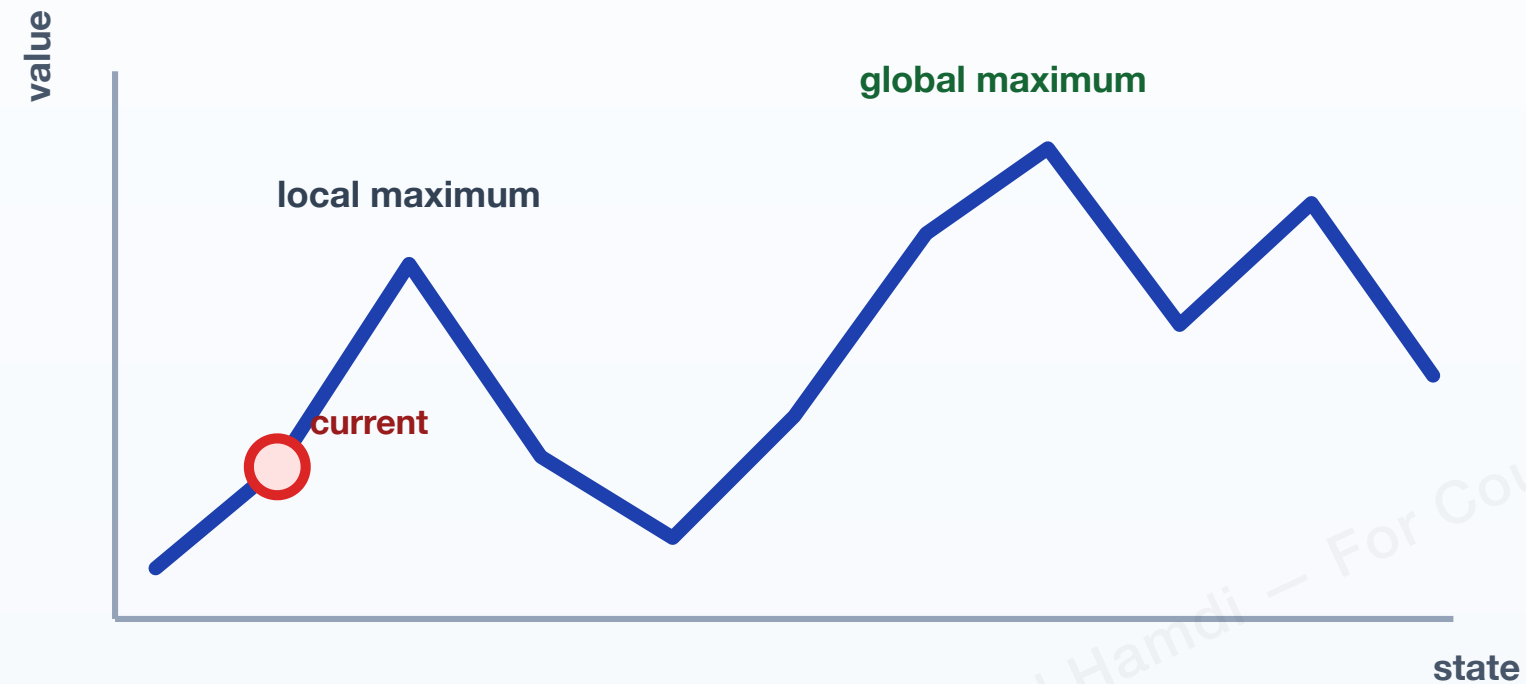
- At high T , worse moves may be accepted.
- At low T , the algorithm behaves more greedily.
- The cooling schedule controls the exploration-exploitation balance.

If cooling is too fast, simulated annealing may behave like hill climbing and get trapped.

Simulated Annealing Walkthrough

Dr. Fayçal Hamdi - Course Use Only - 2026

Controlled randomness can cross valleys



Step 1 — high temperature

Main idea

At high temperature, simulated annealing is exploratory. A randomly selected worse move may still be accepted.

Current state

state x1

Objective value

moderate value

Decision

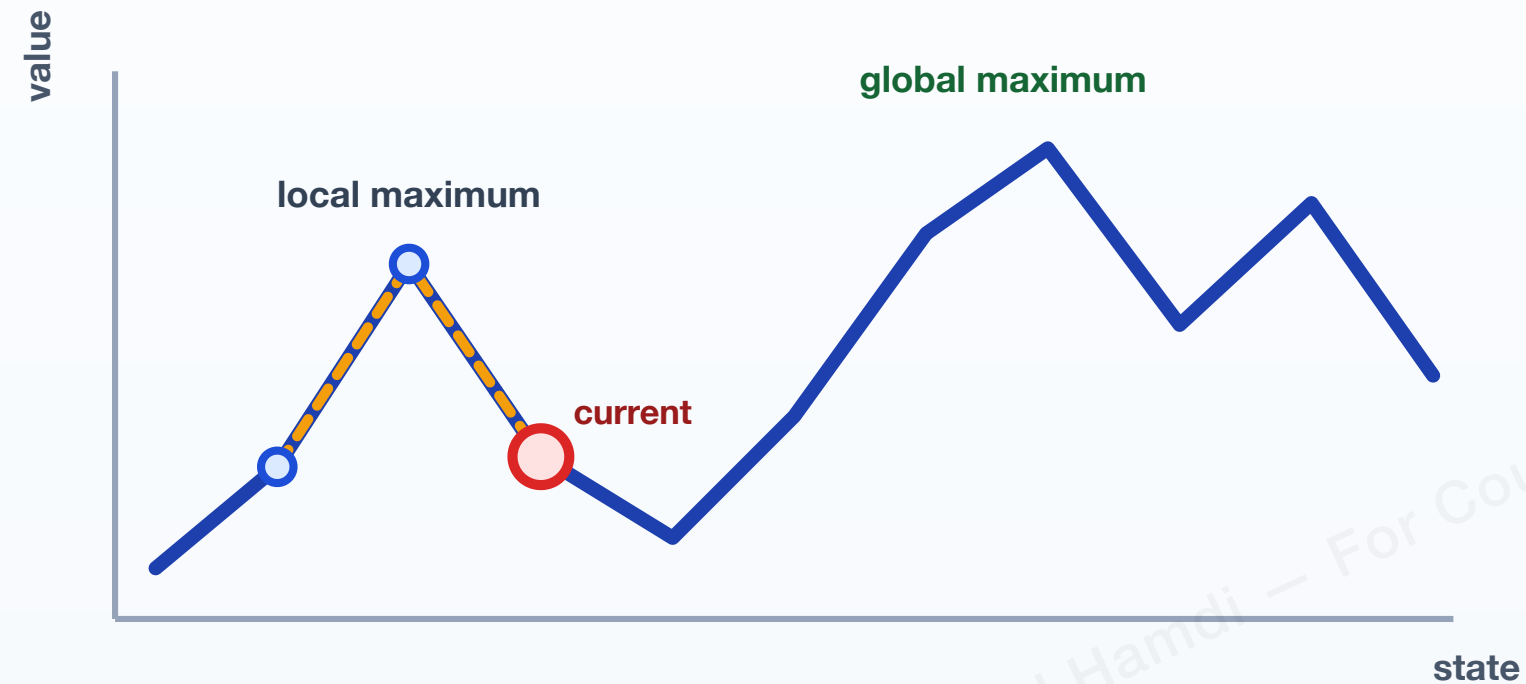
Select a neighbor; accept improvements, and sometimes accept worse moves.

Legend

current state

visited state

Controlled randomness can cross valleys



Step 2 — escape local maximum

Main idea

A worse move may be accepted with a probability that depends on the temperature and the decrease in value.

Current state

state x3

Objective value

temporarily worse

Decision

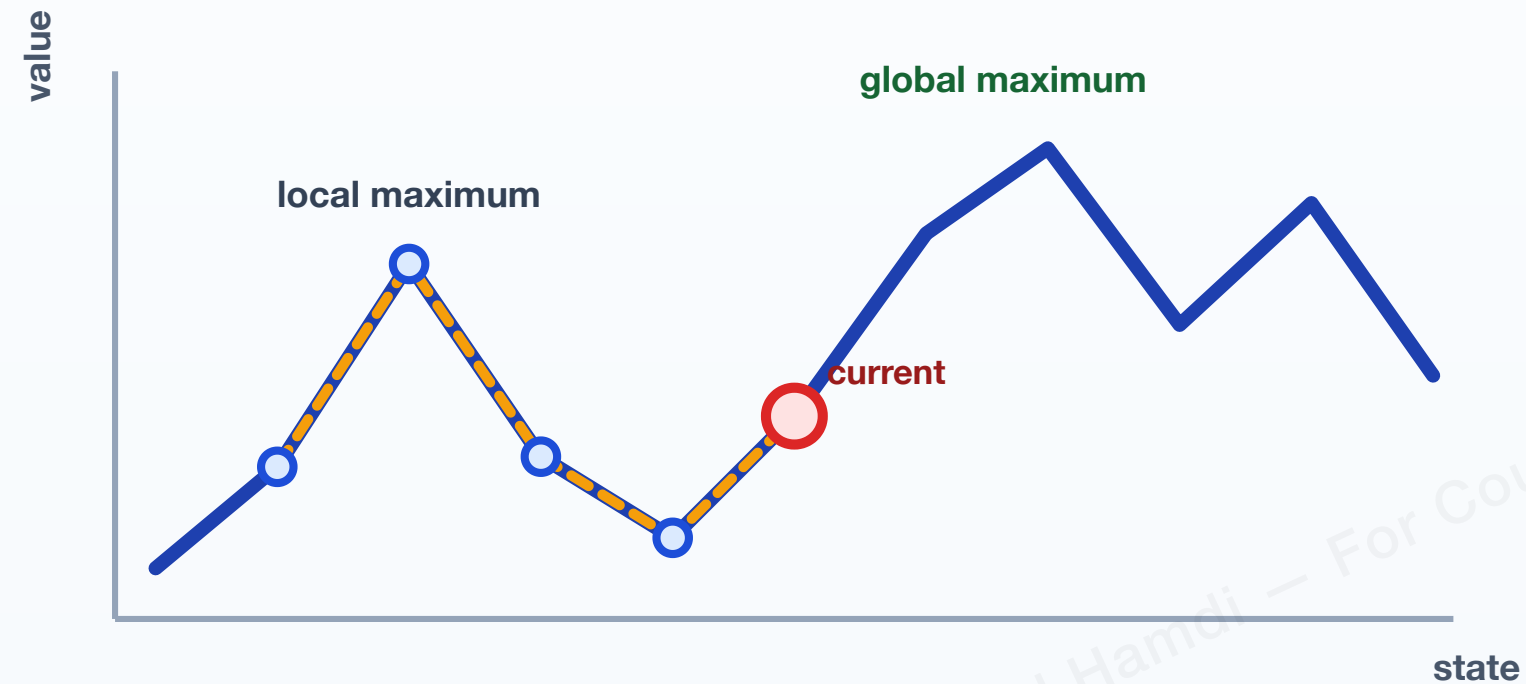
The downhill move can help the search escape a local maximum.

Legend

current state

visited state

Controlled randomness can cross valleys



Step 3 — explore new region

Main idea

While the temperature remains sufficiently high, the search can continue exploring and move into a more promising region.

Current state

state x5

Objective value

improving again

Decision

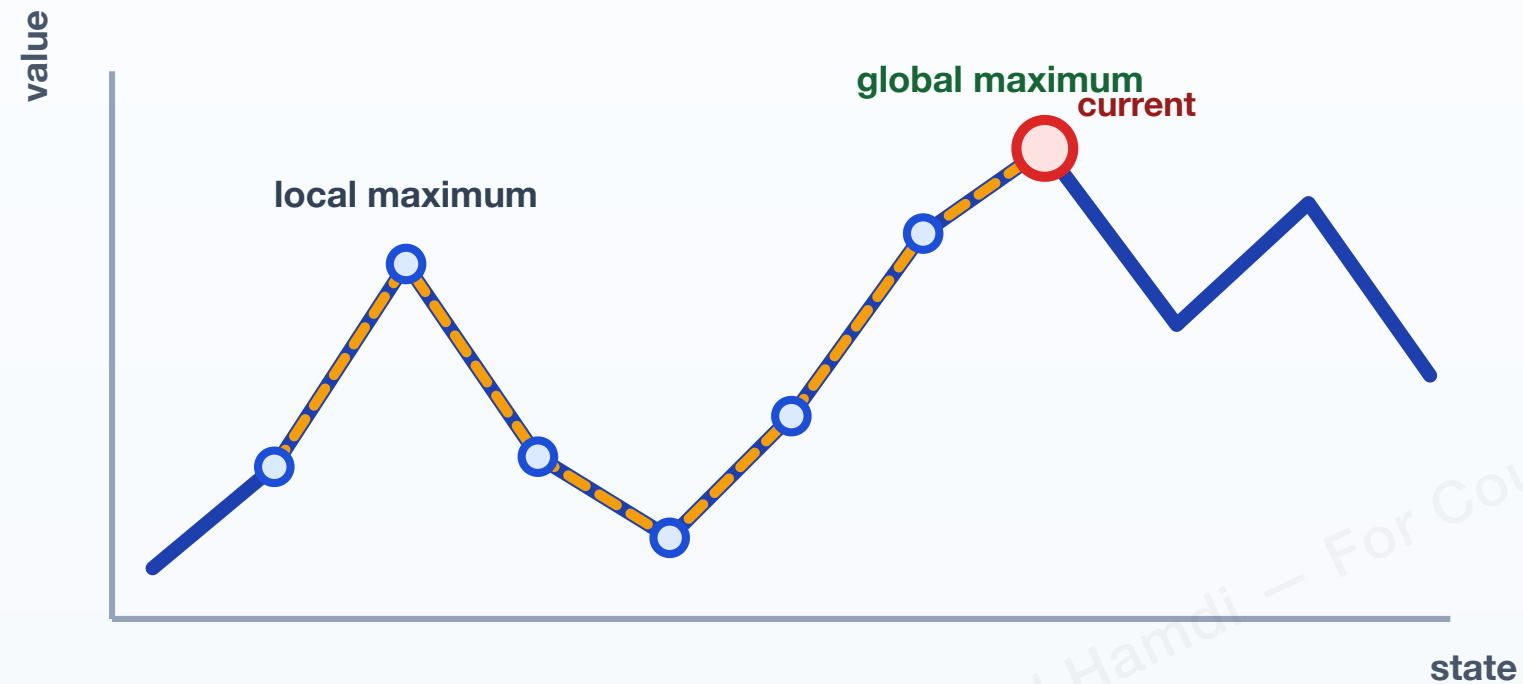
Continue according to the cooling schedule.

Legend

current state

visited state

Controlled randomness can cross valleys



Step 4 – low temperature

Main idea

As the temperature decreases, worse moves become less likely and the search becomes increasingly selective.

Current state

state x7

Objective value

highest value reached in this illustrated run

Decision

As the temperature approaches zero, the search tends to settle near a high-value state.

Legend

current state

visited state

Simulated Annealing — Pseudocode

Dr. Fayçal Hamdi - Course Use Only - 2026

Temperature controls exploration

```
def simulated_annealing(problem, schedule):  
    current = random_initial_state(problem)  
  
    for t in range(1, infinity):  
        T = schedule(t)  
        if T == 0:  
            return current  
  
        next_state = random_neighbor(current)  
        delta = value(next_state) - value(current)  
  
        if delta > 0:  
            current = next_state  
        else:  
            current = next_state with probability  $\exp(\text{delta} / T)$ 
```

💡 The probability of accepting a worse move decreases as the temperature decreases.

Cooling Schedule

A small design choice with large consequences

A cooling schedule defines how T changes over time.

Schedule idea	Example form	Behavior
Fast cooling	$T(t) = \frac{T_0}{t}$	Quickly becomes greedy; may trap early.
Slow cooling	$T(t) = \frac{T_0}{\log(t+1)}$	More exploration; slower convergence.
Exponential cooling	$T(t) = \alpha^t T_0, 0 < \alpha < 1$	Common practical compromise.

A good schedule allows enough early exploration while still converging toward stable solutions later.

Local Beam Search

Keep several states instead of one

Local beam search maintains k current states. At each iteration, it generates successors of all k states and keeps the best k among them.

$$B_{t+1} = \text{best}_k \left(\bigcup_{s \in B_t} N(s) \right)$$

This is not the same as running k independent searches. The candidates share information because the next beam is selected from the combined successor pool.

Local Beam Search Walkthrough

Several candidates are improved together

generation 0



generation 1

generation 2

generation 3

Step 1 — keep multiple states

Main idea

Local beam search starts with k candidate states instead of one. Here, $k = 3$.

Current beam

beam = {42, 55, 31}

Best value

55

Decision

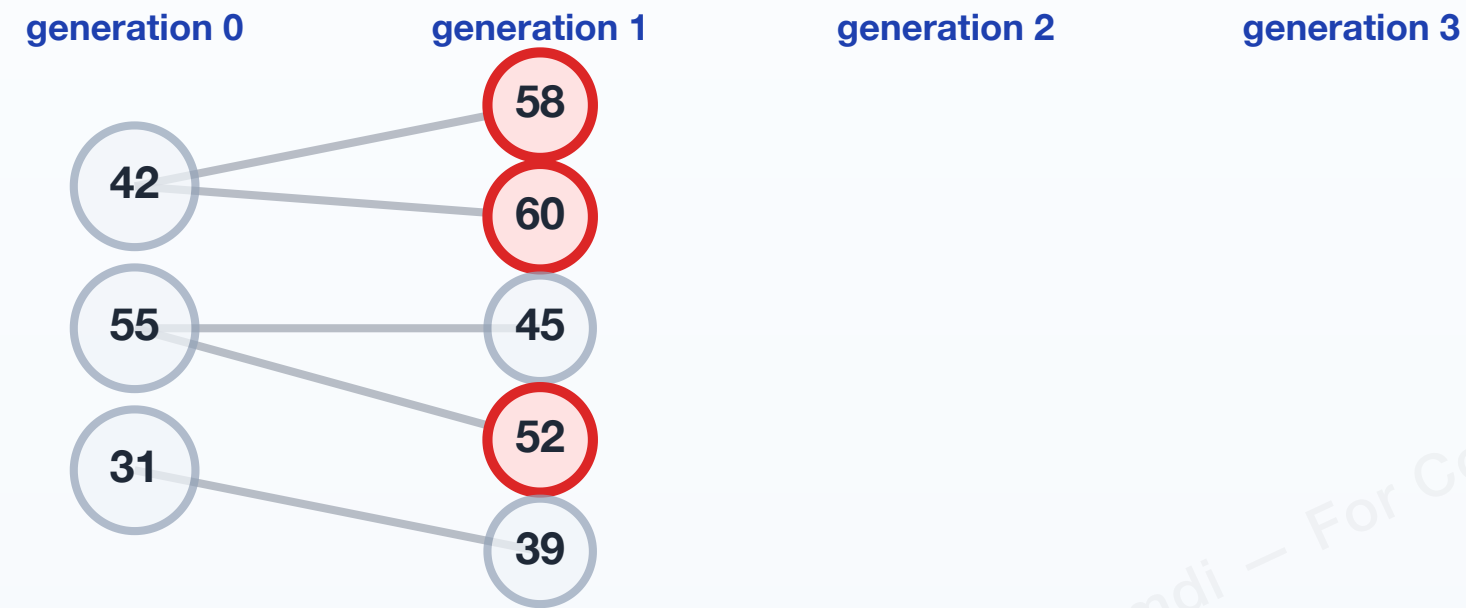
Generate successors from all k beam states.

Legend

retained state

discarded candidate

Several candidates are improved together



Step 2 — select best successors

Main idea

Successors from all beam states are evaluated together. Only the best k candidates survive.

Successor pool

{58, 60, 45, 52, 39}

Current beam

beam = {60, 58, 52}

Decision

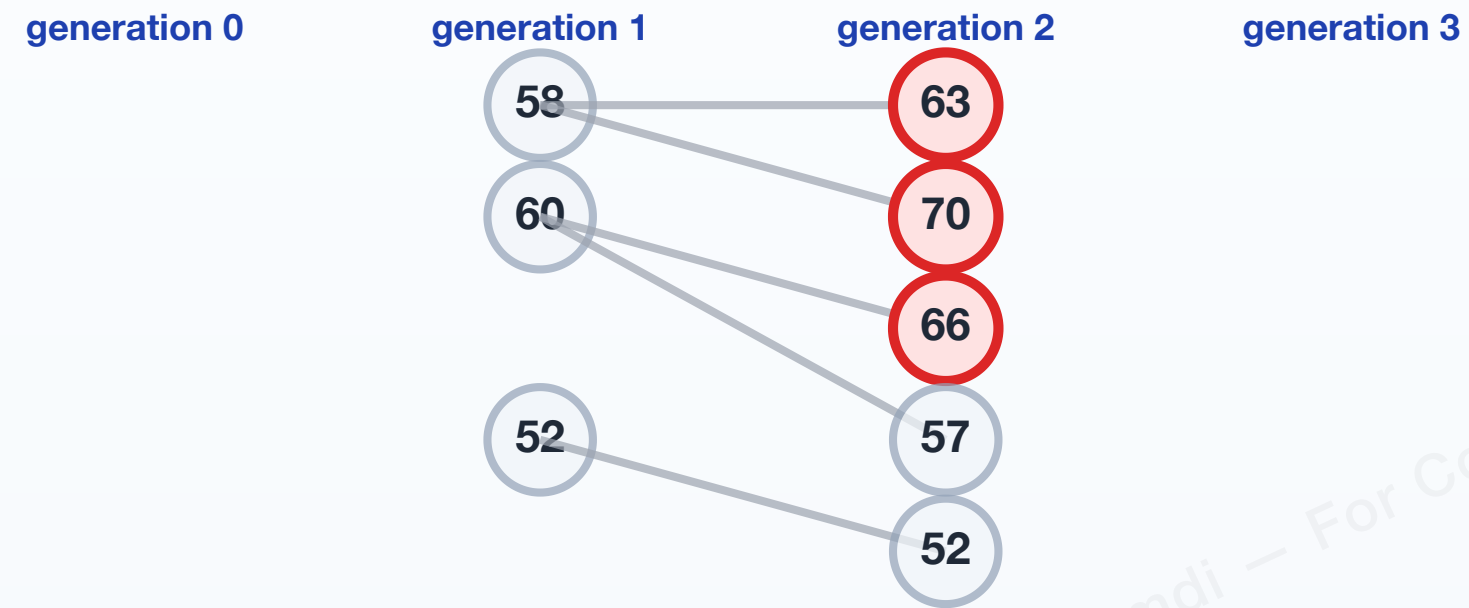
Retain the best $k = 3$ successors and discard the rest.

Legend

retained state

discarded candidate

Several candidates are improved together



Step 3 — share search effort

Main idea

Successors from all beam states compete in one pool, so promising regions can receive more search effort.

Successor pool

{63, 70, 66, 57, 52}

Current beam

beam = {70, 66, 63}

Decision

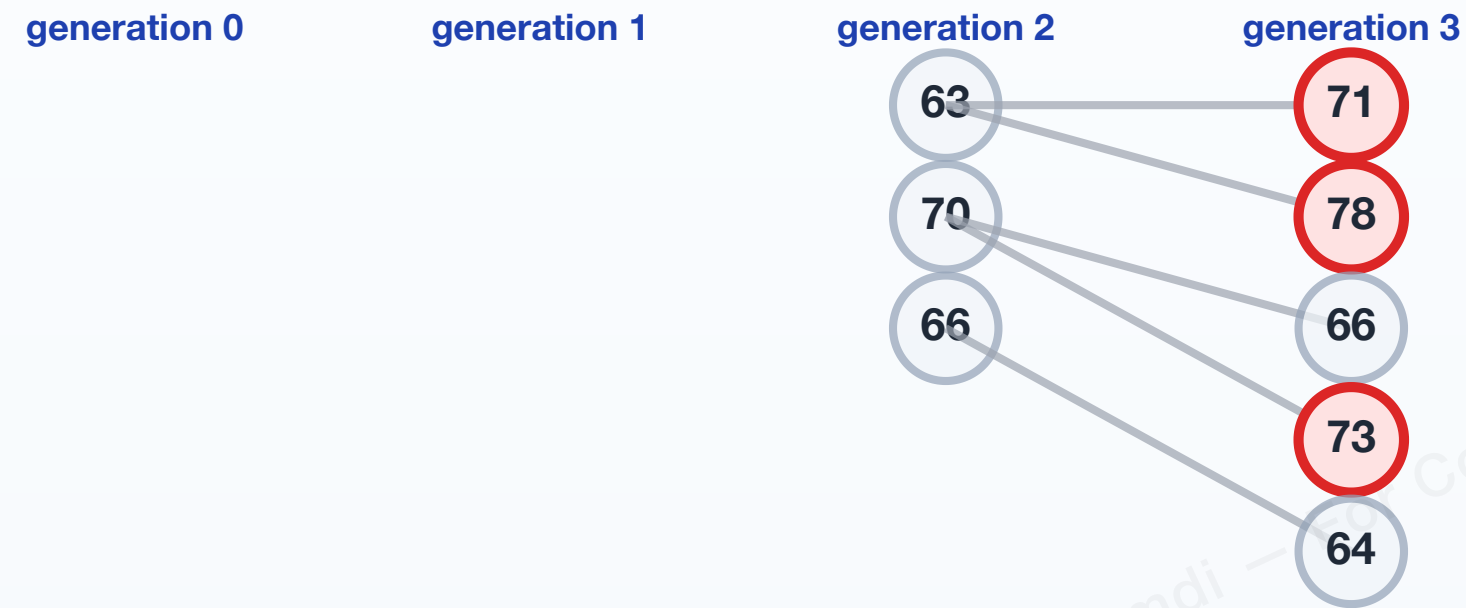
Continue from the strongest k candidates.

Legend

retained state

discarded candidate

Several candidates are improved together



Step 4 — convergence risk

Main idea

As the beam concentrates on promising regions, its states may become too similar and diversity can be lost.

Successor pool

{71, 78, 66, 73, 64}

Current beam

beam = {78, 73, 71}

Decision

Stochastic beam search can help preserve diversity.

Legend

retained state

discarded candidate

Local Beam Search: Strengths and Risks

More candidates can help, but diversity matters

Aspect	Effect
Memory use	Stores k states instead of one, still much less than frontier-based search.
Information sharing	Strong candidates can guide future exploration.
Risk of loss of diversity	All states may cluster in the same region.
Stochastic beam search	Selects successors probabilistically, often preserving more diversity.

💡 Beam search is useful when one local path is too fragile but exhaustive search is too expensive.

Genetic Algorithms

Dr. Fayçal Hamdi — Course Use Only - 2026

Search using a population of encoded candidates

Core components

- **Population:** a set of candidate solutions.
- **Encoding:** representation of candidates as strings, vectors, trees, or other structures.
- **Fitness:** objective value used to rank candidates.
- **Selection:** preferentially choose high-fitness candidates.
- **Crossover:** combine parts of parents.
- **Mutation:** randomly change small parts.

Search behavior

Genetic algorithms explore many candidates in parallel. They are especially useful when the search space is large, structured, and difficult to differentiate analytically.

They can be used for scheduling, feature selection, hyperparameter search, design optimization, and rule discovery.

Genetic Algorithm Walkthrough

Dr. Fayçal Hamdi - Course Use Only - 2026

Selection, crossover, and mutation

candidate 1	1	0	1	1	0	0	fitness = 6.2
candidate 2	1	1	1	0	1	0	fitness = 8.8
candidate 3	0	0	1	1	1	0	fitness = 4.5
candidate 4	1	0	0	1	1	1	fitness = 7.1

Step 1 — initialize and evaluate

Main idea

Initialize a population of encoded candidate solutions and evaluate their fitness.

Current population

four encoded candidate solutions

Fitness

measures the quality of each candidate

Decision

Evaluate the fitness of every candidate.

Legend

candidate

Selection, crossover, and mutation

candidate 1	1	0	1	1	0	0	fitness = 6.2
candidate 2	1	1	1	0	1	0	fitness = 8.8
candidate 3	0	0	1	1	1	0	fitness = 4.5
candidate 4	1	0	0	1	1	1	fitness = 7.1

selected parents

Step 2 — select parents

Main idea

Higher-fitness candidates are more likely to be selected as parents.

Selected parents

candidate 2 and candidate 4

Fitness

8.8 and 7.1

Decision

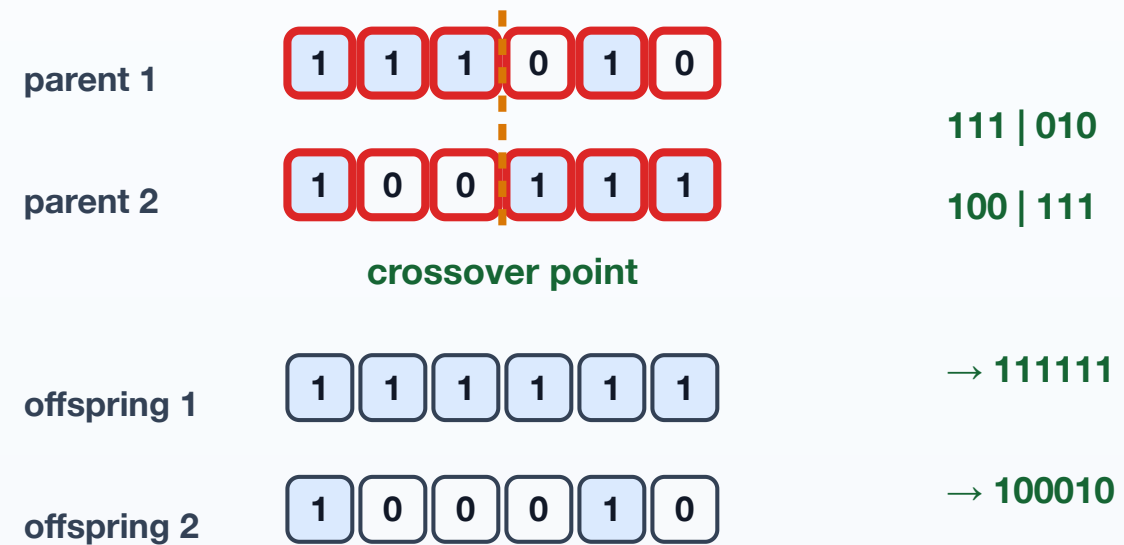
Select parents according to the selection method.

Legend

candidate

parent

Selection, crossover, and mutation



Step 3 — crossover

Main idea

Combine genetic material from the selected parents to produce offspring.

Parents

111|010 and 100|111

Offspring

111111 and 100010

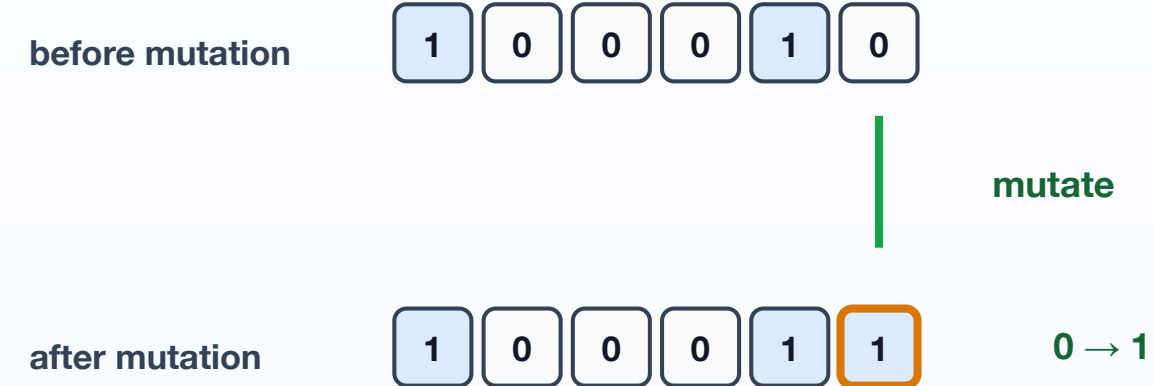
Decision

Apply one-point crossover after gene 3.

Legend

parent offspring

Selection, crossover, and mutation



Step 4 – mutation

Main idea

With a small probability, one or more genes mutate to introduce variation.

Before

offspring = 100010

After

mutated offspring = 100011

Decision

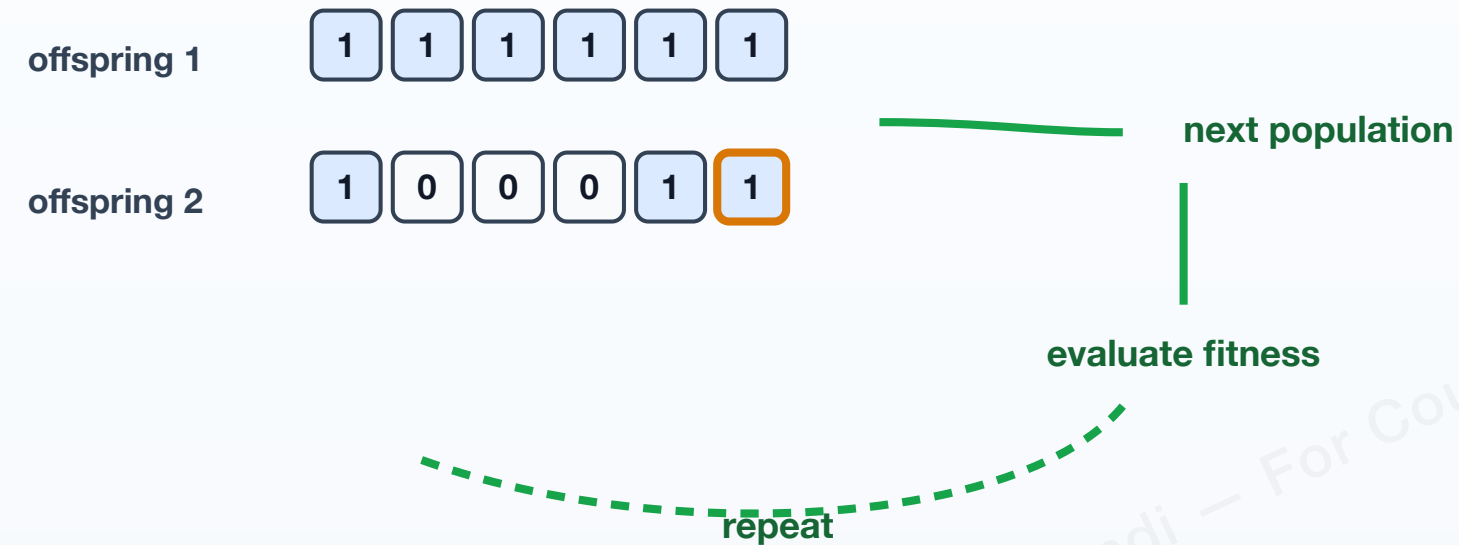
Flip the selected gene with a small mutation probability.

Legend

offspring

mutated gene

Selection, crossover, and mutation



Step 5 — form next generation

Main idea

Offspring contribute to the next population according to the replacement strategy, and the evolutionary cycle repeats.

New offspring

111111 and 100011

Next step

evaluate fitness in the next population

Decision

Repeat selection, crossover, and mutation until the stopping condition is met.

Cycle

evaluate → select → crossover → mutate → repeat

Genetic Algorithm — Pseudocode

Dr. Fayçal Hamdi — Course Use Only - 2026

Evolutionary improvement loop

```
def genetic_algorithm(problem, population_size):  
    population = random_population(population_size)  
  
    while not stopping_condition(population):  
        fitness_values = evaluate(population)  
        parents = select_parents(population, fitness_values)  
        children = []  
  
        for p1, p2 in parent_pairs(parents):  
            child = crossover(p1, p2)  
            child = mutate(child)  
            children.append(child)  
  
        population = select_next_generation(population, children)  
  
    return best_individual(population)
```

The quality of a genetic algorithm depends strongly on representation, fitness design, selection pressure, crossover, mutation rate, and stopping rule.

Local, Global, and Optimal Search Strategies

Interpreting the CLO terminology

In this course context, the three terms can be understood as follows:

Term	Meaning in practice	Examples
Local search	Maintains one or a few current states and improves them using neighborhood information.	Hill climbing, simulated annealing, local beam search, genetic algorithms
Global/systematic search	Explores a broader state-space structure using a frontier or search tree.	BFS, UCS, DFS, A*
Optimal search	Provides conditions under which the returned solution is guaranteed optimal.	UCS with positive costs, A* with admissible or consistent heuristics

💡 A method can be systematic without being efficient, and efficient without being optimal. Algorithm choice depends on the problem requirements.

Performance Comparison

No strategy is universally best

Strategy family	Memory	Typical speed	Optimality guarantee	Main risk	Good use case
BFS / UCS / A*	High	Can be slow	Strong under conditions	Memory explosion	Need complete/optimal paths
Hill climbing	Very low	Often fast	No	Local optima and plateaus	Quick improvement of complete states
Random restart	Low per run	Moderate	No strict guarantee	More computation	Many local optima, cheap evaluations
Simulated annealing	Very low	Moderate	Depends on schedule assumptions	Poor cooling schedule	Escaping local traps
Local beam search	Low to moderate	Often good	No	Loss of diversity	Parallel candidate improvement
Genetic algorithms	Moderate	Problem-dependent	No	Poor representation/fitness	Large combinatorial design spaces

Choosing a Strategy

Dr. Fayçal Hamdi - For Course Use Only - 2026

A practical decision guide

Problem situation	Suitable strategy	Reason
Need the exact least-cost path	UCS or A*	They can provide optimality guarantees under standard conditions.
Need a good configuration, not a path	Local search	The final state matters more than the route to it.
Very large state space and limited memory	Hill climbing or simulated annealing	They store very little.
Many local optima but cheap evaluations	Random-restart hill climbing	Multiple starts improve reliability.
Need controlled exploration of worse moves	Simulated annealing	Temperature allows temporary risk-taking.
Want multiple candidates to cooperate	Local beam search	It shares information among candidate states.
Representation supports recombination	Genetic algorithm	Crossover and mutation can exploit structured encodings.

Worked Example: N-Queens as Local Search

Complete-state formulation

For the N-Queens problem, a local-search formulation can use one queen per column.

Component	Possible definition
State	A complete board with one queen in each column.
Neighbor	Move one queen to another row in the same column.
Objective	Minimize the number of attacking queen pairs.
Goal condition	Objective value equals zero conflicts.

This is a complete-state formulation: every state is a full candidate solution, even if it violates constraints.

Hill climbing can rapidly reduce conflicts, but it may stop at a board where every single queen move fails to improve the number of conflicts.

Worked Example: Travelling Salesperson Problem

Optimization without caring about the path of search

For the Travelling Salesperson Problem, a candidate state can be a complete tour.

Component	Possible definition
State	An ordering of cities representing a tour.
Neighbor	Swap two cities or reverse a segment of the tour.
Objective	Minimize total tour length.
Local optimum	No allowed small modification improves the tour.

💡 The search path is not the travelling route. The search path is the sequence of candidate tours considered by the algorithm.

Mini-Quiz

Dr. Fayçal Hamdi - For Course Use Only - 2026

Check your understanding

1. Why can hill climbing stop before reaching the global optimum?
2. What is the difference between a local optimum and a global optimum?
3. Why does simulated annealing sometimes accept worse moves?
4. How is local beam search different from running several independent hill-climbing searches?
5. Which parts of a genetic algorithm are responsible for exploration and diversity?

Expected insight: local search is powerful because it is memory-efficient, but performance depends heavily on the landscape, neighborhood design, randomness, and stopping rule.

Lecture 3 Summary

Key ideas to retain

Optimization perspective

- Local search focuses on complete candidate states.
- An objective function defines solution quality.
- A neighborhood function defines possible local moves.
- Local optima, plateaus, and ridges affect performance.

Strategy behavior

- Hill climbing is simple and memory-efficient.
- Random restarts reduce dependence on initial state.
- Simulated annealing accepts controlled risk.
- Beam search and genetic algorithms use multiple candidates.
- Optimality guarantees usually require systematic search conditions.

End-of-Lecture Check

Five questions to test your understanding

1. In which types of problems is the search path irrelevant?
2. How does the choice of neighborhood function affect local search performance?
3. Why is random-restart hill climbing often more reliable than a single run?
4. What does the temperature parameter control in simulated annealing?
5. Why does a genetic algorithm need both selection and mutation?

Preparation for next lecture: Review variables, domains, constraints, and examples such as map coloring, Sudoku, scheduling, and N-Queens. Lecture 4 studies Constraint Satisfaction Problems.

Suggested Reading

Primary textbook

S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th edition, Prentice Hall, 2020.

- Chapter 4: Search in Complex Environments
- Focus especially on local search, hill climbing, simulated annealing, local beam search, and evolutionary algorithms.

💡 When reading, pay attention to how each algorithm balances solution quality, memory use, randomness, and guarantees.